

OPTICAM

Esta colección de Postman contiene el conjunto completo de pruebas funcionales realizadas sobre el backend del proyecto web Opticam, una plataforma de comercio electrónico desarrollada bajo una arquitectura cliente-servidor. Su propósito es validar el correcto funcionamiento de los servicios REST expuestos por la API, verificando la comunicación entre el frontend y la capa de negocio del sistema.

La colección ha sido diseñada como una herramienta de apoyo para las actividades de desarrollo, integración, depuración y validación del backend, permitiendo ejecutar pruebas de consumo de endpoints mediante solicitudes HTTP y evaluar las respuestas generadas por el servidor. A través de estas pruebas es posible comprobar el comportamiento de los diferentes módulos del sistema, así como garantizar la integridad de los datos almacenados en la base de datos.

Dentro de esta colección se encuentran organizadas las pruebas correspondientes a los diferentes roles definidos en la aplicación, incluyendo Administrador y Repartidor, además de los endpoints públicos relacionados con el catálogo de productos. Cada solicitud permite validar operaciones CRUD (Create, Read, Update y Delete), cambios de estado, autenticación de usuarios, control de acceso basado en roles y gestión de recursos asociados al comercio electrónico.

Las pruebas implementadas verifican aspectos técnicos fundamentales como:

- Autenticación mediante JSON Web Token (JWT).
- Autorización basada en roles y permisos.
- Consumo de servicios REST utilizando los métodos HTTP GET, POST, PUT, PATCH y DELETE.
- Validación de parámetros enviados por URL.
- Validación de datos enviados mediante cuerpos de solicitud en formato JSON.
- Gestión de respuestas generadas por el servidor.
- Persistencia y recuperación de información desde la base de datos.
- Integridad de las relaciones entre entidades como usuarios, categorías, subcategorías, productos, proveedores y pedidos.
- Control de estados de registros activos e inactivos.
- Actualización dinámica de inventario y stock de productos.

La estructura de la colección sigue la organización lógica del backend de Opticam, agrupando las solicitudes según los módulos funcionales del sistema. Esto facilita el mantenimiento, la identificación de errores, la ejecución de pruebas individuales y la validación integral de los procesos de negocio implementados en la aplicación.

Esta documentación sirve como evidencia técnica del funcionamiento de la API y como guía para desarrolladores, evaluadores y futuros mantenedores del proyecto, permitiendo comprender de manera clara las rutas disponibles, los mecanismos de autenticación utilizados y las operaciones soportadas por el sistema.

INVENTARIO

Esta carpeta agrupa los endpoints relacionados con la gestión de productos y categorías del sistema, permitiendo

consultar, buscar y filtrar productos, así como administrar su creación, actualización y eliminación. También incluye la gestión de categorías, marcas y colores disponibles en el inventario.

El módulo maneja imágenes de productos almacenadas en Cloudinary, además de filtros avanzados para facilitar la búsqueda de productos por características como precio, marca, material o categoría.

PRODUCTOS(PUBLICOS)

Esta carpeta agrupa los endpoints relacionados con la consulta de productos disponibles para los usuarios sin necesidad de permisos administrativos. Permite visualizar el catálogo completo de productos, ver productos destacados, consultar detalles individuales, aplicar filtros avanzados, buscar por texto y explorar productos por categoría, marca, color o características específicas.

GET Listar todos los productos

`:/api/inventario/productos`

Permite obtener el listado completo de productos disponibles en el sistema, incluyendo imágenes optimizadas.

Método y ruta:

GET `:/api/inventario/productos`

Comportamiento esperado:

Retorna todos los productos con imagen en URL y thumbnail.

Respuesta exitosa:

Plain Text

```
{  "success": true,  "count": 10,  "productos": []}
```

Errores comunes:

500 si ocurre un error interno del servidor.

GET Productos destacados

`:/api/inventario/productos/destacados`

Permite obtener los últimos productos destacados del sistema.

Método y ruta:

GET `:/api/productos/destacados`

Comportamiento esperado:

Retorna los productos más recientes o destacados.

Errores comunes:

500 error interno.

GET Obtener producto por ID

`:/api/inventario/productos/3`

Permite consultar la información detallada de un producto específico.

Método y ruta:

GET `:/api/productos/:id`

Parámetros:

- id (requerido)

Errores comunes:

404 si el producto no existe.

GET Buscar productos

`:/api/inventario/productos/buscar?q=montura`

StartFragment

Permite buscar productos por texto (nombre, marca, descripción).

Método y ruta:

GET `:/api/productos/buscar?q=texto`

Errores comunes:

400 si no se envía el query.

EndFragment

PARAMS

q

montura

GET Filtrar productos

`:/api/inventario/productos/filtros?precio_min=100000&precio_max=300000&marca=Rayban`

Permite aplicar filtros avanzados sobre el catálogo.

Método y ruta:

GET `:/api/productos/filtrar`

Query opcional:

- precio_min
- precio_max
- marca
- color
- material
- id_categoria
- q (búsqueda)

PARAMS

precio_min	100000
precio_max	300000
marca	Rayban

GET Productos por categoría

`:/api/inventario/productos/categoria/1`

Permite obtener productos filtrados por categoría.

Método y ruta:

GET `:/api/productos/categoria/:id_categoria`

GET Productos por marca

`:/api/inventario/productos/marca/Rayban`

StartFragment

Permite obtener productos filtrados por marca.

Método y ruta:

GET `:/api/productos/marca`

GET Listar marcas

`:/api/inventario/marcas`

Método y ruta:

GET `:/api/productos/marca`

GET Listar colores

`:/api/inventario/colores`

StartFragment

Permite listar todos los colores disponibles.

Método y ruta:

GET `:/api/productos/colores`

EndFragment

CATEGORIAS(PUBLICO)

Esta carpeta agrupa los endpoints destinados a la consulta de categorías de productos disponibles para todos los usuarios. Permite visualizar las categorías del sistema y consultar su información básica para facilitar la navegación y organización del catálogo de productos.

GET Listar categorías

`:/api/inventario/categorias`

Permite obtener todas las categorías del sistema.

Método y ruta:

GET `:/api/categorias`

GET Obtener categoría por ID

`:/api/inventario/categorias/2`

Permite consultar una categoría específica.

Método y ruta:

GET `:/api/categorias/:id`

Errores comunes:

404 si no existe.

PRODUCTOS(ADMIN)

Esta carpeta agrupa los endpoints destinados a la administración completa del inventario de productos, permitiendo crear, actualizar y eliminar productos, gestionar imágenes en Cloudinary, así como mantener la información del catálogo actualizada incluyendo precio, marca, color, material y categoría.

POST Crear producto

`:/api/inventario/productos`

Permite crear un nuevo producto en el inventario.

Método y ruta:

POST `:/api/inventario/productos`

Body requerido:

- `id_categoria` (requerido)
- `nombre` (requerido)
- `precio` (requerido)

- imagen (file)

Body opcional:

- descripcion
- marca
- material
- color

Comportamiento esperado:

Crea un producto y lo guarda con imagen en Cloudinary.

Errores comunes:

400 si faltan campos obligatorios

400 si no se envía imagen

500 error interno

EndFragment

AUTHORIZATION Bearer Token

Token <token>

HEADERS

Body formdata

id_categoria	3
nombre	Gafas Polarizadas
descripcion	Protección UV400
marca	Rayban
precio	320000
material	Polycarbonato
color	Azul
imagen	

PUT Actualizar producto

`:/api/inventario/productos/31`

Permite actualizar la información de un producto existente.

Método y ruta:

PUT `:/api/inventario/productos/:id`

Body opcional:

- id_categoria
- nombre
- descripcion
- marca
- precio
- material
- color
- imagen (file opcional)

Comportamiento esperado:

Actualiza el producto y reemplaza imagen si se envía una nueva.

Errores comunes:

404 producto no encontrado

500 error interno

EndFragment

AUTHORIZATION Bearer Token

Token `<token>`

Body formdata

marca Oakley

DELETE Eliminar producto

`:/api/inventario/productos/31`

Permite eliminar un producto del sistema.

Método y ruta:

DELETE `:/api/inventario/productos/:id`

Comportamiento esperado:

Elimina el producto y su imagen de Cloudinary.

Errores comunes:

404 si el producto no existe

500 error interno

EndFragment

AUTHORIZATION Bearer Token

Token <token>

CATEGORIAS(ADMIN)

Esta carpeta agrupa los endpoints relacionados con la administración de categorías del sistema, permitiendo crear nuevas categorías, actualizarlas y eliminarlas, asegurando la correcta organización del inventario de productos dentro del sistema.

POST Crear categoría

`:/api/inventario/categorias`

Permite crear una nueva categoría de productos.

Método y ruta:

POST `:/api/admin/categorias`

Body requerido:

- tipo_categoria

Body opcional:

- descripcion

AUTHORIZATION Bearer Token

Token

{{supabase_service_role_api_key_0lin}}

Body raw (json)

json

```
{
  "tipo_categoria": "GAFAS DE SOL",
  "descripcion": "Gafas de sol polarizadas"
}
```

PUT Actualizar categoría

:/api/inventario/categorias/3

Permite modificar una categoría existente.

Método y ruta:

PUT `:/api/inventario/categorias/:id`

AUTHORIZATION Bearer Token

Token

{{admin}}

Body raw (json)

json

```
{
  "descripcion": "Gafas de sol polarizadas, Gafas de sol deportivas, Gafas de sol classicas"
}
```

DELETE Eliminar categoría

:/api/inventario/categorias/3

Permite eliminar una categoría del sistema.

Método y ruta:

DELETE `:/api/inventario/categorias/:id`

Errores comunes:

404 si no existe.

AUTHORIZATION Bearer Token

Token	{{admin}}
-------	-----------

CHATBOT

Esta carpeta agrupa los endpoints relacionados con el asistente virtual del sistema, permitiendo a los usuarios enviar mensajes para recibir respuestas automáticas basadas en detección de intención, consultar preguntas frecuentes (FAQ) y buscar información dentro de las mismas. El chatbot no almacena información en base de datos, funcionando como un sistema de respuesta inmediata basado en reglas.

POST Enviar mensaje al chatbot

`:/api/chatbot/mensaje`

Permite al usuario enviar un mensaje al chatbot y recibir una respuesta automática basada en la detección de intención del mensaje.

Método y ruta:

POST `:/api/chatbot/mensaje`

Body requerido:

Plain Text

`mensaje: string` (not `null`)

Comportamiento esperado:

Plain Text

```
{  "success": true,  "mensaje_usuario": "¿Cómo hago un pedido?",  "respuesta_chatbot": "Para hacer un pedido, primero debes iniciar sesión en tu cuenta y luego ir a la sección de pedidos."}
```

Errores comunes:

400 si el mensaje está vacío o no se envía.

500 si ocurre un error interno del servidor.

HEADERS

Content-Type application/json

Body raw (json)

json

```
{  
  "mensaje": "ayuda"  
}
```

GET Obtener todas las FAQ

: /api/chatbot/faq

Permite consultar todas las preguntas frecuentes disponibles del sistema.

Método y ruta:

GET

Comportamiento esperado:

Plain Text

```
{  "success": true,  "count": 5,  "faqs": []}
```

Errores comunes:

500 error interno del servidor.

GET Buscar FAQ por texto

: /api/chatbot/faq/buscar?q=envío

Permite buscar preguntas frecuentes filtrando por texto.

Método y ruta:

GET

Comportamiento esperado:

Plain Text

```
{  "success": true,  "query": "pedido",  "count": 2,  "resultados": []}
```

Errores comunes:

400 si no se envía el parámetro `q`.

500 si ocurre un error interno del servidor.

PARAMS

q	envío
---	-------

PEDIDOS

Esta carpeta agrupa los endpoints relacionados con la gestión de pedidos dentro del sistema, permitiendo a los clientes crear pedidos con productos y fórmulas médicas asociadas, consultar su historial, ver detalles específicos y cancelar pedidos bajo ciertas condiciones. También incluye funcionalidades administrativas para gestionar estados del pedido, fechas de entrega, validación de pagos parciales, y control del flujo completo hasta la entrega final.

El módulo maneja la integración con productos, fórmulas médicas, pagos y cálculo automático de costos de envío y fechas estimadas de entrega.

PEDIDO(CLIENTE)

Esta carpeta agrupa los endpoints relacionados con la gestión de pedidos por parte del cliente, permitiendo crear pedidos con productos y fórmulas médicas, consultar su historial, ver el detalle de cada pedido y cancelar aquellos que aún no hayan avanzado a estados de procesamiento o envío. También permite visualizar la información completa del pedido, incluyendo productos, costos, estado y fecha estimada de entrega.

POST Crear pedido

`:/api/pedidos`

Permite al cliente crear un pedido con productos, opcionalmente asociando una fórmula médica aprobada, calculando automáticamente costos de envío, subtotal y fecha estimada de entrega.

Método y ruta:

POST `:/api/pedidos`

Body requerido:

Plain Text

```
{
  "id_formula": 8,
  "direccion_entrega": "Avenida 3 #78-90, Cali",
  "productos": [
    {
      "id_producto": 1,
      "cant_productos": 1
    }
  ]
}
```

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "message": "Pedido creado exitosamente",
  "data": {}
}
```

Errores comunes:

- 400 si falta la dirección de entrega.
- 400 si no se envían productos.
- 404 si un producto no existe.
- 403 si la fórmula no pertenece al usuario.
- 404 si la fórmula no está aprobada.
- 500 error interno.

AUTHORIZATION Bearer Token

Token

<token>

Body raw (json)

json

```
{
  "id_formula": 8,
```

```
{
  "direccion_entrega": "Avenida 3 #78-90, Cali",
  "productos": [
    {
      "id_producto": 1,
      "cant_productos": 1
    }
  ]
}
```

GET Mis pedidos

`:/api/pedidos/mis-pedidos`

Permite al cliente consultar todos sus pedidos con sus productos asociados.

Método y ruta:

GET `:/api/pedidos/mis-pedidos`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "count": 2,
  "data": []
}
```

Errores comunes:

- 401 si no está autenticado.
- 500 error interno.

AUTHORIZATION Bearer Token

Token `<token>`

PUT Cancelar pedido

`:/api/pedidos/14/cancelar`

Permite al cliente cancelar un pedido siempre que no esté en estados avanzados como Enviado, Entregado o En Proceso.

Método y ruta:

PUT `:/api/pedidos/:id/cancelar`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "message": "Pedido cancelado exitosamente",
  "data": {}
}
```

Errores comunes:

- 400 si el pedido no puede cancelarse por su estado.
- 403 si no pertenece al usuario.
- 404 si no existe.
- 500 error interno.

AUTHORIZATION Bearer Token

Token

<token>

PEDIDO(ADMIN)

Esta carpeta agrupa los endpoints destinados a la administración de pedidos dentro del sistema, permitiendo consultar todos los pedidos, filtrarlos por estado, actualizar su estado en el flujo logístico, gestionar fechas estimadas de entrega, validar pedidos abonados y controlar el proceso completo hasta la entrega final del producto. Además, incluye endpoints para obtener estadísticas generales del comportamiento de los pedidos.

PUT Marcar como LISTO

`:/api/pedidos/2/listo`

Permite al administrador marcar un pedido como **"Listo"** únicamente cuando el pedido se encuentra en estado **"Abonado"** y cuenta con un pago confirmado del 50%. Este estado habilita al cliente a realizar el pago del 50% restante antes del despacho del pedido.

Método y ruta:

PUT `:/api/pedidos/:id/listo`

Comportamiento esperado:

Plain Text


```
{
  "success": true,
  "message": "Pedido marcado como LISTO. El cliente puede pagar el 50% restante.",
  "data": {}
}
```

Errores comunes:

- 404 si el pedido no existe.
- 400 si el pedido no está en estado "Abonado".
- 400 si no tiene un abono del 50% confirmado.
- 400 si ya fue pagado al 100%.
- 500 si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token <token>

GET Todos los pedidos

:/api/pedidos/admin/todos

Permite al administrador consultar todos los pedidos registrados en el sistema.

Método y ruta:

GET `:/api/admin/todos`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "count": 10,
  "data": []
}
```

Errores comunes:

- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

GET Pedidos por estado

`:/api/pedidos/admin/estado/Pagado`

Método y ruta:

GET `:/api/pedidos/admin/estado/pagado`

Plain Text

```
{  "success": true,  "count": 0,  "data": []}
```

Errores comunes:

400 si el estado es inválido.

500 error interno.

AUTHORIZATION Bearer Token

Token `<token>`

PUT Actualizar estado

`:/api/pedidos/1/estado`

Permite al administrador cambiar el estado del pedido dentro del flujo logístico.

Método y ruta:

PUT `:/api/pedidos/:id/estado`

Body requerido:

Plain Text

```
{  "estado": "En Proceso"}
```

Comportamiento esperado:

Plain Text

```
{  "success": true,  "message": "Estado actualizado a: Enviado",  "data": {}}
```

Errores comunes:

- 400 estado inválido.
- 404 si el pedido no existe.
- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

Body raw (json)

json

```
{  
  "estado": "En Proceso"  
}
```

PUT Actualizar fecha estimada

`:/api/pedidos/5/fecha-estimada`

Permite al administrador modificar la fecha estimada de entrega de un pedido.

Método y ruta:

PUT `:/api/pedidos/:id/fecha-estimada`

Body requerido:

Plain Text

`fecha_estimada: string (YYYY-MM-DD)`

Comportamiento esperado:

Plain Text

```
{ "success": true, "message": "Fecha estimada de entrega actualizada", "data": {} }
```

Errores comunes:

- 400 si falta la fecha.
- 404 si el pedido no existe.

500 error interno

- 500 error interno.

Body raw (json)

json

```
{  
  "fecha_estimada": "2026-07-15"  
}
```

GET Estadísticas

`:/api/pedidos/admin/estadisticas`

Permite al administrador ver métricas generales del sistema de pedidos.

Método y ruta:

GET `:/api/pedidos/estadisticas`

Comportamiento esperado:

Plain Text

```
{  "success": true,  "data": {}}
```

Errores comunes:

- 500 error interno.

AUTHORIZATION Bearer Token

Token

<token>

GET Ver detalle de pedido

`:/api/pedidos/14`

Permite al cliente o administrador consultar el detalle completo de un pedido, incluyendo información del cliente, productos asociados y datos de la fórmula médica (si aplica), validando permisos de acceso según el usuario autenticado.

Método y ruta:

GET `:/api/pedidos/:id`

Comportamiento esperado:

Plain Text

```
{  "success": true,  "data": {    "pedido": {      "id_pedido": 1,      "fecha_pedido": "2026-01-01T00:00:00.000Z"    }  }
```

Errores comunes:

404 si el pedido no existe.

403 si el usuario no tiene permisos para verlo.

401 si el usuario no está autenticado.

500 si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token `<token>`

PAGOS

Esta carpeta agrupa los endpoints relacionados con la gestión de pagos de pedidos, permitiendo al cliente realizar pagos parciales (50%) o completos (100%), consultar los pagos asociados a un pedido y verificar el estado del saldo. También incluye funcionalidades administrativas para consultar pagos, ver estadísticas, filtrar por fechas y gestionar la confirmación o rechazo de pagos mediante integración con el sistema externo Bold.

PAGO(CLIENTE)

Esta carpeta agrupa los endpoints relacionados con la gestión de pagos por parte del cliente, permitiendo realizar pagos parciales (50%) o completos (100%) de sus pedidos, consultar el historial de pagos por pedido y verificar el estado del saldo pendiente en tiempo real.

POST Crear pago

`:/api/pagos`

Permite al cliente registrar un pago parcial (50%) o total (100%) para un pedido, validando el estado del pedido, el monto y evitando pagos duplicados o inconsistentes.

Método y ruta:

POST `:/api/pagos`

Body requerido:

Plain Text

```
{
  "id_pedido": 6,
  "eleccion_pago": "50%",
  "monto": 155000
}
```

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "message": "Pago registrado exitosamente. Esperando confirmación de Bold",
  "data": {}
}
```

Errores comunes:

- 400 si faltan campos requeridos (id_pedido, eleccion_pago, monto).
- 400 si eleccion_pago no es 50% o 100%.
- 404 si el pedido no existe.
- 400 si el monto excede el total del pedido.
- 400 si el pedido ya está completamente pagado.
- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

HEADERS

Content-Type application/json

Body raw (json)

json

```
{
  "id_pedido": 6,
  "eleccion_pago": "50%",
  "monto": 155000
}
```

```
    "eleccion_pago": 50%,
    "monto": 155000
  }
```

GET Ver pago por ID

`:/api/pagos/:id`

Permite consultar un pago específico por su ID.

Método y ruta:

GET `:/api/pagos/:id`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 404 si el pago no existe.
- 500 error interno.

AUTHORIZATION Bearer Token

Token `<token>`

PATH VARIABLES

id	1
----	---

GET Ver saldo pendiente

`:/api/pagos/pedido/1/saldo`

Permite al cliente consultar el estado de pago de un pedido, incluyendo el total del pedido, lo pagado hasta el momento, el saldo pendiente y el estado actual del pago (sin pago, abonado o pagado completo)

El estado pendiente y el estado actual del pago (con pago, devuelto o pagado completo).

Método y ruta:

GET `:/api/pagos/pedido/:Id/sald`

Comportamiento esperado:

Plain Text

```
{  "success": true,  "data": {    "total_pedido": 100000,    "total_pagado": 50000,    "saldo_
```

Errores comunes:

- 404 si el pedido no existe.
- 500 si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token <token>

GET Pagos de un pedido

`:/api/pagos/pedido/1`

Permite consultar todos los pagos asociados a un pedido específico, incluyendo el detalle de cada pago, el total acumulado pagado y si el pedido ya fue pagado completamente.

Método y ruta:

GET `:/api/pagos/pedido/:pedidoId`

Comportamiento esperado:

Plain Text

```
{  "success": true,  "count": 2,  "total_pagado": 50000,  "pagado_completo": false,  "data": [
```

Errores comunes:

- 404 si el pedido no existe o no tiene pagos asociados.
- 500 si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token <token>

PAGO(BOLD)

Esta carpeta agrupa los endpoints de integración con el sistema de pagos externo Bold, encargados de confirmar o rechazar transacciones mediante webhooks, actualizando automáticamente el estado de los pagos y sincronizando el estado del pedido según el resultado de la transacción.

PUT Confirmar pago

`:/api/pagos/12/confirmar`

Permite confirmar un pago recibido desde Bold y actualizar automáticamente el estado del pago y del pedido.

Método y ruta:

POST `:/api/pagos/:id/confirmar`

Comportamiento esperado:

Plain Text

```
{  "success": true,  "message": "Pago completo confirmado. El pedido está en estado PAGADO",
```

Errores comunes:

- 404 si el pago no existe.
 - 400 si el pago ya está confirmado.
 - 500 error interno.
-

PUT Rechazar pago

`:/api/pagos/12/rechazar`

Permite rechazar un pago desde Bold cuando la transacción falla.

Método y ruta:

POST `:/api/pagos/:id/rechazar`

Body requerido:

Plain Text

```
motivo: string (opcional)
```

Comportamiento esperado:

Plain Text

```
{  "success": true,  "message": "Pago rechazado",  "motivo": "No especificado",  "data": {}}
```

Errores comunes:

- 404 si el pago no existe.
- 400 si el pago ya fue confirmado.
- 500 error interno.

Body raw (json)

json

```
{  "motivo": "Fondos insuficientes"}
```

PAGO(ADMIN)

Esta carpeta agrupa los endpoints destinados a la administración de los pagos dentro del sistema, permitiendo consultar todos los pagos registrados, ver detalles por ID, obtener estadísticas generales, analizar pagos por rango de fechas y monitorear el comportamiento de los ingresos por método de pago.

GET Listar pagos

:/api/pagos

Permite al administrador consultar todos los pagos registrados en el sistema.

Método y ruta:

GET `:/api/pagos`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "count": 10,
  "data": []
}
```

Errores comunes:

- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

GET Estadísticas

:/api/pagos/estadisticas

Permite consultar estadísticas generales de pagos y su distribución por método (50% / 100%).

Método y ruta:

GET

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

HEADERS

Content-Type application/json

GET Pagos por rango de fechas

`:/api/pagos/rango-fechas?fechaInicio=2025-01-01&fechaFin=2025-01-31`

Permite filtrar pagos según un rango de fechas.

Método y ruta:

GET `:/api/admin/pagos/rango?fechaInicio=YYYY-MM-DD&fechaFin=YYYY-MM-DD`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "count": 5,
  "data": []
}
```

Errores comunes:

- 400 si faltan fechas.
- 500 error interno.

AUTHORIZATION Bearer Token

Token `<token>`

PARAMS

fechaInicio 2025-01-01

fechaFin 2025-01-31

FORMULAS

Esta carpeta agrupa los endpoints encargados de la gestión de fórmulas ópticas dentro del sistema, permitiendo a los clientes subir, consultar y eliminar sus fórmulas, así como a los administradores revisarlas, aprobarlas, asignarles un costo y gestionar su estado para su posterior procesamiento.

FORMULA(CLIENTE)

Esta carpeta agrupa los endpoints relacionados con la gestión de fórmulas por parte del cliente, permitiendo subir nuevas fórmulas, consultar sus fórmulas registradas, eliminar aquellas en estado pendiente y verificar el estado de aprobación y el costo asignado por el administrador.

POST Subir fórmula

`:/api/formulas`

Permite que un cliente suba una fórmula médica con su respectiva imagen para revisión del administrador.

Método y ruta:

POST `:/api/formulas`

Body requerido:

Plain Text

```
{
  condicion: string (valores permitidos: DALTONISMO, ASTIGMATISMO, MIOPIA, BAJA VISION)
  imagen_formula: file (not null)
  observaciones: string
}
```

Comportamiento esperado:

Si la validación es correcta, se registra la fórmula asociada al usuario autenticado y responde con:

Plain Text

```
{
  "success": true,
  "message": "Fórmula subida exitosamente. Esperando revisión del administrador",
  "data": {}
}
```

Errores comunes:

- 400 si no se envía el campo `condicion`.
- 400 si no se envía la imagen de la fórmula (`imagen_formula`).
- 400 si la `condicion` no es válida (debe ser DALTONISMO, ASTIGMATISMO, MIOPIA o BAJA VISION).
- 500 si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token

tokens

Body formdata

condicion	BAJA VISION
observaciones	Paciente con baja visión severa - 20/200
imagen	

GET Mis fórmulas

:/api/formulas/mis-formulas

Permite al cliente consultar todas sus fórmulas médicas registradas en el sistema, incluyendo su estado, costo e imagen asociada.

Método y ruta:

GET `:/api/formulas/mis-formulas`

Comportamiento esperado:

Si el usuario está autenticado, se obtienen todas las fórmulas asociadas al cliente y se responde con:

Plain Text

```
{
  "success": true,
  "count": 2,
  "data": []
}
```

Errores comunes:

- 401 si el usuario no está autenticado o no se envía token.
- 500 si ocurre un error interno del servidor al consultar las fórmulas.

AUTHORIZATION Bearer Token

Token `<token>`

DELETE Eliminar formula

:/api/formulas/8

Permite al cliente eliminar una fórmula médica propia siempre que se encuentre en estado “Pendiente”, incluyendo la eliminación de la imagen asociada en el almacenamiento.

Método y ruta:

DELETE `:/api/formulas/:id`

Comportamiento esperado:

Si la validación es correcta, la fórmula es eliminada del sistema y su imagen asociada es eliminada del almacenamiento (Cloudinary). Se responde con:

Plain Text

```
{
  "success": true,
  "message": "Fórmula eliminada exitosamente"
}
```

Errores comunes:

- 404 si la fórmula no existe.
- 403 si la fórmula no pertenece al usuario autenticado.
- 400 si la fórmula no está en estado "Pendiente" (solo se pueden eliminar fórmulas pendientes).
- 401 si el usuario no está autenticado.
- 500 si ocurre un error interno del servidor o falla en el proceso de eliminación.

AUTHORIZATION Bearer Token

Token	<token>
-------	---------

GET Ver fórmula por ID

:/api/formulas/8

Permite al cliente o administrador consultar una fórmula médica específica por su ID, validando su existencia y permisos de acceso.

Método y ruta:

GET `:/api/formulas/:id`

Comportamiento esperado:

Si la fórmula existe y el usuario tiene permisos para consultarla, se retorna la información de la fórmula junto con su imagen procesada:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 404 si la fórmula no existe.
- 403 si el usuario no tiene permisos para ver la fórmula.
- 401 si el usuario no está autenticado.
- 500 si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token <token>

GET Verificar fórmula aprobada

`:/api/formulas/8/verificar`

Permite al cliente verificar si una fórmula está aprobada.

Método y ruta:

GET `:/api/formulas/:id/verificar`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 404 fórmula no encontrada.
- 403 sin permisos.
- 401 no autenticado.
- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

FORMULA(ADMIN)

Esta carpeta agrupa los endpoints encargados de la gestión administrativa de las fórmulas, permitiendo al administrador visualizar todas las fórmulas registradas, revisar las pendientes, aprobar o rechazar fórmulas, asignarles un costo y actualizar su estado dentro del sistema.

GET Todas las fórmulas

`:/api/formulas/admin/todas`

Permite al administrador ver todas las fórmulas registradas en el sistema.

Método y ruta:

GET `:/api/admin/todas`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "count": 10,
  "data": []
}
```

Errores comunes:

- 500 si ocurre un error interno.

AUTHORIZATION Bearer Token

Token `<token>`

GET Fórmulas pendientes

`:/api/formulas/admin/pendientes`

Permite al administrador ver únicamente las fórmulas en estado pendiente.

Método y ruta:

GET `:/api/formulas/admin/pendientes`

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "count": 3,
  "data": []
}
```

Errores comunes:

- 500 error interno.

AUTHORIZATION Bearer Token

Token

<token>

PUT Asignar precio

`:/api/formulas/8/precio`

Permite al administrador asignar costo y opcionalmente cambiar el estado de la fórmula.

Método y ruta:

PUT `:/api/formulas/:id/precio`

Body requerido:

Plain Text

```
{
  "costo": 150000,
  "estado": "Aprobado"
}
```

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "message": "Precio asignado exitosamente. Estado: Aprobado",
  "data": {}
}
```

Errores comunes:

- 400 si falta el costo.
- 400 si el costo es negativo.
- 404 si la fórmula no existe.

AUTHORIZATION Bearer Token

Token

<token>

Body raw (json)

json

```
{
  "costo": 150000,
  "estado": "Aprobado"
}
```

PUT Cambiar estado

:/api/formulas/4/estado

Permite cambiar el estado de una fórmula.

Método y ruta:

PUT `:/api/formulas/:id/estado`

Body requerido:

Plain Text

```
{
  "estado": "Rechazado"
}
```

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "message": "Estado actualizado a: Aprobado",
}
```

```
"data": {}  
}
```

Errores comunes:

- 400 estado inválido.
- 404 fórmula no encontrada.
- 500 error interno.

AUTHORIZATION Bearer Token

Token <token>

Body raw (json)

json

```
{  
  "estado": "Rechazado"  
}
```

DISTRIBUCIONES

Esta carpeta organiza los endpoints de gestión de distribuciones según los roles del sistema, separando las operaciones del administrador y del repartidor en el proceso de entrega.

DISTRIBUCION(ADMIN)

Esta carpeta contiene los endpoints administrativos relacionados con la gestión de distribuciones y entregas dentro del sistema. Incluye solicitudes para asignar pedidos, consultar todas las distribuciones, cancelar entregas y visualizar distribuciones externas.

POST Asignar pedido

:/api/distribucion

Permite asignar un pedido a un repartidor en Bogotá o a una distribuidora externa si el pedido es fuera de la ciudad, validando el estado del pedido y evitando asignaciones duplicadas.

Método y ruta:

metodo y ruta.
POST :/api/distribucion

Body requerido:

Plain Text

```
{  
  "id_pedido": 123,  
  "id_usuario": 45,  
  "observaciones": "Entregar en recepción"  
}
```

Comportamiento esperado:

Si la validación es correcta, responde con:

Plain Text

```
{  
  "success": true,  
  "message": "Pedido asignado exitosamente al repartidor / distribuidora externa",  
  "data": {}  
}
```

Errores comunes:

- 400 si faltan campos obligatorios id_pedido o id_usuario.
- 400 si el pedido no está en estado "Pagado".
- 400 si el pedido ya tiene una distribución activa.
- 404 si el pedido no existe.
- 404 si el repartidor no existe, no está activo o no tiene rol REPARTIDOR.
- 404 si no se encuentra un administrador para distribución externa.
- 404 si el administrador asignado no existe o está inactivo.
- 500 si ocurre un error interno del servidor.

HEADERS

Authorization	token admin
Content-Type	application/ecmascript

Body raw (json)

json

```
{
  "id_pedido": 1,
  "id_usuario": 5,
  "observaciones": "Entrega prioritaria"
}
```

GET Todas las distribuciones

`:/api/distribucion/admin/todas`

Permite obtener el listado de todas las distribuciones registradas en el sistema, incluyendo información básica del pedido y del repartidor asignado.

Método y ruta:

GET `:/api/distribucion/admin/todas`

Comportamiento esperado:

Si la consulta es exitosa, responde con:

Plain Text

```
{
  "success": true,
  "count": 10,
  "data": []
}
```

Errores comunes:

500 si ocurre un error interno del servidor.

PUT Cancelar entrega

`:/api/distribucion/admin/:id/cancelar`

Permite cancelar una entrega asociada a una distribución, siempre que no haya sido marcada como entregada previamente, registrando una observación opcional.

Método y ruta:

PUT `:/api/admin/distribuciones/:id/cancelar`

Body requerido:

Plain Text

```
{
  "observacion": "Cliente no disponible"
}
```

Comportamiento esperado:

Si la validación es correcta, responde con:

Plain Text

```
{
  "success": true,
  "message": "Entrega cancelada exitosamente",
  "data": {}
}
```

Errores comunes:

- 404 si la distribución no existe.
- 400 si la distribución ya está en estado "ENTREGADO".
- 500 si ocurre un error interno del servidor.

PATH VARIABLES

id

GET Ver distribuciones externas

:/api/distribucion/admin/externas

Permite obtener todas las distribuciones externas asociadas a pedidos fuera de Bogotá, enriqueciendo la información con los datos del cliente, el pedido y el administrador encargado de la distribución.

Método y ruta:

GET :/api/admin/distribuciones/externas

Comportamiento esperado:

Si la consulta es exitosa, responde con:

Plain Text

```
{
  "success": true,
  "count": 5,
  "data": []
}
```

```
}
```

Errores comunes:

404 si no se encuentra un administrador activo.

500 si ocurre un error interno del servidor.

DISTRIBUCION(REPA)

Esta carpeta contiene los endpoints que usa el repartidor para consultar, iniciar y completar entregas, así como revisar su historial y detalles de distribución.

GET Pedidos pendientes

`:/api/distribucion/pendientes`

Permite al repartidor autenticado consultar todos los pedidos pendientes asignados a él, incluyendo información básica del pedido como dirección, total y cliente.

Método y ruta:

GET `:/api/distribucion/pendientes`

Comportamiento esperado:

Si la consulta es correcta, responde con:

Plain Text

```
{
  "success": true,
  "count": 3,
  "data": []
}
```

Errores Comunes:

- 401 si el usuario no está autenticado.
- 500 si ocurre un error interno del servidor.

GET Pedidos en entrega

`:/api/distribucion/en-entrega`

Permite al repartidor autenticado consultar los pedidos que actualmente se encuentran en estado de entrega, mostrando

información del pedido, dirección y cliente asociado.

Método y ruta:

```
GET :/api/distribucion/en-entrega
```

Comportamiento esperado:

Si la consulta es correcta, responde con:

Plain Text

```
{
  "success": true,
  "count": 2,
  "data": []
}
```

Errores comunes:

- 401 si el usuario no está autenticado.
- 500 si ocurre un error interno del servidor.

PATCH Iniciar entrega

`:/api/distribucion/:id/iniciar`

Permite al repartidor iniciar una entrega asignada, cambiando el estado del pedido de **PENDIENTE** a **EN_ENTREGA**, siempre que el pedido le pertenezca y esté en el estado correcto.

Método y ruta:

```
PATCH :/api/repartidor/distribuciones/:id/iniciar
```

Comportamiento esperado:

Si la validación es correcta, responde con:

Plain Text

```
{
  "success": true,
  "message": "Entrega iniciada exitosamente",
  "data": {}
}
```

Errores comunes:

- 401 si el usuario no está autenticado.
- 403 si el repartidor no tiene permiso para iniciar esta entrega

- 403 si el repartidor no tiene permiso para iniciar esta entrega.
- 404 si la distribución no existe.
- 400 si la distribución no está en estado "PENDIENTE".
- 500 si ocurre un error interno del servidor.

PATH VARIABLES

id

PATCH Marcar como entregado

/api/distribucion/:id/entregar

Permite al repartidor marcar una entrega como finalizada, actualizando el estado de la distribución a **ENTREGADO**, siempre que la entrega esté en curso y le pertenezca al usuario autenticado.

Método y ruta:

PUT ./api/distribucion/:id/entregar

Comportamiento esperado:

Si la validación es correcta, responde con:

Plain Text

```
{
  "success": true,
  "message": "Pedido marcado como entregado exitosamente",
  "data": {}
}
```

Errores comunes:

- 401 si el usuario no está autenticado.
- 403 si el repartidor no tiene permiso para marcar esta entrega.
- 404 si la distribución no existe.
- 400 si la distribución no está en estado "EN_ENTREGA".
- 500 si ocurre un error interno del servidor.

PATH VARIABLES

id

GET Historial de entregas

`:/api/distribucion/historial`

Permite al repartidor consultar el historial de entregas completadas, mostrando información detallada de cada pedido entregado, incluyendo dirección, cliente y fechas relevantes.

Método y ruta:

`GET :/api/distribucion/historial`

Comportamiento esperado:

Si la consulta es correcta, responde con:

Plain Text

```
{
  "success": true,
  "count": 4,
  "data": []
}
```

Errores comunes

- 401 si el usuario no está autenticado.
- 500 si ocurre un error interno del servidor.

GET Ver detalle distribución

`:/api/distribucion/:id`

Permite consultar el detalle completo de una distribución específica, incluyendo información del pedido y del repartidor asignado, con control de acceso según el usuario autenticado.

Método y ruta:

`GET :/api/distribucion/:id`

Comportamiento esperado:

Si la consulta es correcta, responde con:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 401 si el usuario no está autenticado.
- 403 si el usuario no tiene permiso para ver esta distribución.
- 404 si la distribución no existe.
- 500 si ocurre un error interno del servidor.

PATH VARIABLES

id

REPORTES

Esta carpeta agrupa los endpoints encargados de generar análisis y reportes del sistema, permitiendo obtener información estadística de ventas, productos, clientes, repartidores, inventario y desempeño general del negocio.

GET Ventas por período

`/api/reportes/ventas-periodo`

Permite obtener un reporte de ventas filtrado por un rango de fechas, mostrando el detalle diario y un resumen general del período. Solo se consideran pedidos con estado diferente a **CANCELADO**.

Ejemplo (query params):

Plain Text

```
GET /api/reportes/ventas-periodo?fecha_inicio=2026-01-01&fecha_fin=2026-01-31
```

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 400 si no se envían `fecha_inicio` o `fecha_fin`.
- 500 si ocurre un error interno del servidor.

GET Productos más vendidos

`:/api/reportes/productos-mas-vendidos`

Permite obtener un listado de los productos más vendidos, ordenados de mayor a menor según la cantidad vendida. Opcionalmente puede filtrarse por rango de fechas.

Ejemplos (Query params):

Plain Text

```
GET /api/reportes/productos-mas-vendidos?limite=10&fecha_inicio=2026-01-01&fecha_fin=2026-01-31
```

Notas:

- `limite` es opcional (por defecto 10)
- `fecha_inicio` y `fecha_fin` son opcionales, pero deben enviarse juntos si se desea filtrar por período

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": []
}
```

Errores comunes:

- 500 si ocurre un error interno del servidor
- 400 puede ocurrir si los parámetros de fecha son inválidos (aunque no está validado explícitamente en el código)

GET Desempeño de repartidores

`:/api/reportes/desempeno-repartidores`

Permite obtener el desempeño de los repartidores, mostrando métricas como pedidos asignados, valor total de entregas y promedio de ventas. Opcionalmente puede filtrarse por rango de fechas.

Ejemplos (Query params):

Plain Text

```
GET /api/reportes/desempeno-repartidores?fecha_inicio=2026-01-01&fecha_fin=2026-01-31
```

Notas:

- fecha_inicio y fecha_fin son opcionales, pero deben enviarse juntos si se desea filtrar por período
- Si no se envían fechas, se obtiene el desempeño general de todos los repartidores

Comportamiento esperado:

Plain Text

```
{  
  "success": true,  
  "data": []  
}
```

Errores comunes:

- 500 si ocurre un error interno del servidor

GET Estado de pedidos

`/api/reportes/estado-pedidos`

Permite obtener un reporte del estado de los pedidos, mostrando métricas como cantidad de pedidos, monto total, promedio, mínimo y máximo por cada estado. Opcionalmente puede filtrarse por rango de fechas.

Ejemplos (Query params):

Plain Text

```
GET /api/reportes/estado-pedidos?fecha_inicio=2026-01-01&fecha_fin=2026-01-31
```

Notas:

- fecha_inicio y fecha_fin son opcionales, pero deben enviarse juntos si se desea filtrar por período
- Si no se envían fechas, se obtiene el reporte general de todos los pedidos

Comportamiento esperado:

Plain Text

```
{  
  "success": true,  
  "data": {}  
}
```

GET Clientes frecuentes

`/api/reportes/clientes-frecuentes`

Permite obtener un listado de los clientes más frecuentes, ordenados por el total gastado. Muestra métricas como número de pedidos, total gastado, promedio de gasto, compra mayor y menor. Opcionalmente puede filtrarse por rango de fechas.

Ejemplos (Query params):

Plain Text

```
GET /api/reportes/clientes-frecuentes?limite=10&fecha_inicio=2026-01-01&fecha_fin=2026-01-31
```

Notas:

- limite es opcional (por defecto 10)
- fecha_inicio y fecha_fin son opcionales, pero deben enviarse juntos si se desea filtrar por período
- Solo se consideran pedidos con estado diferente a CANCELADO

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": []
}
```

Errores comunes:

- 500 si ocurre un error interno del servidor

GET Resumen general

`/api/reportes/resumen-general`

Permite obtener un resumen general del negocio con métricas globales de usuarios, productos, pedidos, fórmulas e ingresos, incluyendo información del último mes y evolución de ingresos por mes.

Ejemplos (Query params):

Plain Text

GET /api/reportes/resumen-general

Notas:

- Este endpoint no requiere parámetros obligatorios
- No utiliza filtros por fecha en la consulta principal
- Los cálculos de “último mes” se realizan automáticamente con base en la fecha actual
- Excluye pedidos en estado CANCELADO

Comportamiento esperado:

Plain Text

```
{  
  "success": true,  
  "data": {}  
}
```

Errores comunes:

- 500 si ocurre un error interno del servidor

GET Ventas por categoría

/api/reportes/ventas-categoria

Permite obtener un reporte de ventas agrupado por categoría de productos, mostrando métricas como pedidos, unidades vendidas, ingresos y precio promedio por categoría.

Ejemplos (Query params):

Plain Text

GET /api/reportes/ventas-categoria?fecha_inicio=2026-01-01&fecha_fin=2026-01-31

Notas:

- fecha_inicio y fecha_fin son opcionales, pero deben enviarse juntos si se desea filtrar por período
- Si no se envían fechas, se obtiene el reporte general de todas las categorías
- Excluye pedidos en estado CANCELADO

Comportamiento esperado:

Plain Text


```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 500 si ocurre un error interno del servidor

GET Análisis de fórmulas

`/api/reportes/analisis-formulas`

Permite obtener un análisis de las fórmulas registradas en el sistema, mostrando la distribución por condición, costos asociados y la tendencia mensual de creación de nuevas fórmulas.

Ejemplos (Query params):

Plain Text

GET `/api/reportes/analisis-formulas`

Notas:

- Este endpoint no requiere parámetros
- Solo considera fórmulas en estado ACTIVO
- La tendencia mensual se calcula sobre los últimos 6 meses

Comportamiento esperado:

Plain Text

```
{
  "success": true,
  "data": {}
}
```

Errores comunes:

- 500 si ocurre un error interno del servidor

USUARIOS

Esta carpeta agrupa los endpoints relacionados con la gestión de usuarios dentro del sistema. Incluye operaciones de autenticación, así como funcionalidades diferenciadas por rol para administración y clientes, permitiendo registrar, consultar y gestionar la información de los usuarios según sus permisos.

USUARIO(ADMIN)

Esta carpeta contiene los endpoints que usa el administrador para registrar repartidores, listar usuarios, ver usuarios por ID, actualizar su información y eliminar cuentas dentro del sistema.

POST RegistrarRepa

`:/api/usuarios/repartidores`

Permite registrar un nuevo repartidor desde el panel de administración, asignándole automáticamente el rol **REPARTIDOR** y registrando la información de su vehículo.

Método y ruta:

POST `:/api/usuarios/repartidores`

Body requerido:

Plain Text

```
{
  "nombre_completo": "Juan Pérez",
  "telefono": "3001234567",
  "fecha_nacimiento": "1998-05-15",
  "documento": "1234567890", "ciudad":
  "Bogotá", "direccion": "Calle 123 #45-67",
  "email": "juan.perez@email.com",
  "contrasena": "Password123*",
  "vehiculo": {
    "tipo": "MOTO",
    "modelo": "NKD 125",
```

Comportamiento esperado:

Si la información es válida y el registro se realiza correctamente, responde con:

Plain Text

```
{
  "success": true,
  "message": "Repartidor registrado exitosamente".
```

```
"data": { }
```

Errores comunes:

- **400** si falta alguno de los campos requeridos o no se envía el objeto `vehículo`.
- **400** si el correo electrónico ya se encuentra registrado.
- **400** si el documento ya se encuentra registrado.
- **400** si la placa del vehículo ya se encuentra registrada.
- **500** si el rol **REPARTIDOR** no existe en la base de datos.
- **500** si ocurre un error interno del servidor.

AUTHORIZATION Bearer Token

Token `{{supabase_service_role_api_key_0lin}}`

Body raw (json)

json

```
{
  "nombre_completo": "Carlos Repartidor",
  "telefono": "3119876543",
  "fecha_nacimiento": "1988-03-20",
  "documento": 987654321,
  "ciudad": "Medellín",
  "direccion": "Carrera 45 #67-89",
  "email": "carlos@email.com",
  "contrasena": "MiContraseña456",
  "vehiculo": {
    "tipo": "MOTOCICLETA",
    "modelo": "Honda CB190",
    "placa": "XYZ789",
    "color": "Negro"
  }
}
```

GET ObtenerRepartidores

`:/api/usuarios/repartidores`

Permite consultar la lista de todos los repartidores registrados en el sistema, incluyendo sus datos personales, roles y la información de su vehículo.

Método y ruta:

`GET : /api/usuarios/repartidores`

GET :/api/usuarios/repartidores

Comportamiento esperado:

Responde con la lista de repartidores ordenada alfabéticamente por `nombre_completo`.

GET ObtenerRepaFiltro

`:/api/usuarios/repartidores/buscar?nombre=Carlos`

Permite consultar los repartidores registrados aplicando filtros opcionales por nombre, ciudad, estado o placa del vehículo.

Parámetros de consulta (Query Params):

- `nombre` (null): Filtra por nombre completo (búsqueda parcial).
- `ciudad` (null): Filtra por ciudad (búsqueda parcial).
- `estado` (null): Filtra por estado del repartidor.
- `placa` (null): Filtra por placa del vehículo (búsqueda parcial).

Ejemplo de consulta:

Plain Text

GET :/api/usuarios/repartidores?nombre=Juan&ciudad=Bogotá&estado=ACTIVO&placa=ABC

PARAMS

nombre

Carlos

PUT ActualizarRepa

`:/api/usuarios/repartidores/22`

Permite actualizar la información de un repartidor registrado, incluyendo sus datos personales y la información de su vehículo.

Método y ruta:

PUT :/api/usuarios/repartidores/:id

Parámetro requerido:

- `id` : Identificador del repartidor.

Body requerido:

Plain Text

```
{
  "nombre_completo": "Juan Pérez",
  "telefono": "3001234567",
  "fecha_nacimiento": "1998-05-15",
  "documento": "1234567890",
  "ciudad": "Bogotá",
  "direccion": "Calle 123 #45-67",
  "email": "juan.perez@email.com",
  "estado": "ACTIVO",
  "vehiculo": {
    "tipo": "MOTO",
```

Comportamiento esperado:

Si el repartidor existe y la información es válida, responde con:

Plain Text

```
{
  "success": true,
  "message": "Repartidor actualizado exitosamente",
  "data": { }
}
```

Errores comunes:

- **400** si la placa del vehículo ya se encuentra registrada por otro usuario.
- **404** si el repartidor no existe.

PATCH ActualizarEstadoRepa

`:/api/usuarios/repartidores/22/estado`

Permite cambiar el estado de un repartidor registrado en el sistema.

Método y ruta:

`PATCH :/api/usuarios/repartidores/:id/estado`

Parámetro requerido:

- `id`: Identificador del repartidor.

Body requerido:

Plain Text

```
{  "estado": "ACTIVO"}
```

Valores permitidos para `estado`****:

- `ACTIVO`
- `INACTIVO`
- `SUSPENDIDO`

Comportamiento esperado:

Si el repartidor existe y el estado enviado es válido, responde con:

Plain Text

```
{  "success": true,  "message": "Estado del repartidor actualizado a ACTIVO",  "data": {    "id": 1,    "nombre_completo": "Juan Pérez",    "estado": "ACTIVO"  }  }
```

Errores comunes:

- **400** si el campo `estado` no se envía o su valor no es uno de los permitidos (`ACTIVO` `INACTIVO` o `SUSPENDIDO`).
- **404** si el repartidor no existe.

DELETE EliminarRepa

`:/api/usuarios/repartidores/22`

Permite eliminar un repartidor registrado en el sistema, junto con la información asociada a su vehículo y la relación de roles.

Método y ruta:

`DELETE :/api/usuarios/repartidores/:id`

Parámetros requeridos:

- `id` : Identificador del repartidor a eliminar.

Comportamiento esperado:

Si el repartidor existe y la eliminación se realiza correctamente, responde con:

Plain Text

```
{  
  "success": true,  
  "message": "Repartidor eliminado correctamente."  
}
```

Errores comunes:

- **404** si el repartidor no existe o el usuario indicado no tiene el rol **REPARTIDOR**.

USUARIO(CLIENTE)

Esta carpeta contiene los endpoints dirigidos al cliente para su registro y acceso al sistema. Su propósito principal es gestionar las operaciones relacionadas con la creación de cuentas, el inicio de sesión y las acciones básicas de autenticación necesarias para que el usuario cliente pueda ingresar y usar la plataforma.

POST Registrar

`:/api/usuarios/registro`

Permite registrar un cliente en el sistema en formato JSON. El usuario se crea con estado **ACTIVO**, se le asigna automáticamente el rol **CLIENTE** y se genera un token de autenticación.

Método y ruta:

`POST :/api/usuarios/registro`

Body requerido:

Plain Text

```
{  
  "nombre_completo": "Juan Pérez",  
  "telefono": "3001234567",  
  "fecha_nacimiento": "1998-05-15",  
  "documento": "1234567890",  
  "ciudad": "Bogotá",  
  "direccion": "Calle 123 #45-67",  
  "email": "juan.perez@email.com",  
  "contrasena": "Password123*"  
}
```

Comportamiento esperado:

Si la validación es correcta, responde con:

Plain Text

```
{
  "success": true,
  "message": "Cliente registrado exitosamente",
  "data": {}
}
```

Errores comunes:

- **400** si faltan campos obligatorios.
- **400** si el email ya está registrado.
- **400** si el documento ya está registrado.
- **500** si ocurre un error interno del servidor.

Body raw (json)

json

```
{
  "nombre_completo": "Shariht",
  "telefono": "3001234738",
  "fecha_nacimiento": "1990-05-15",
  "documento": 123456345,
  "ciudad": "Bogotá",
  "direccion": "Calle 123 #45-55",
  "email": "shariht@email.com",
  "contrasena": "12345678"
}
```

POST IniciarSesion

:/api/auth/login

Permite autenticar a un usuario en el sistema mediante email y contraseña. El sistema valida las credenciales, verifica que el usuario esté en estado **ACTIVO** y, si la autenticación es correcta, genera un token de acceso.

Método y ruta:

POST :/api/auth/login

Body requerido:

Plain Text

```
{
```



```
{  
  "email": "juan.perez@email.com",  
  
  "contrasena": "Password123*"  
}
```

Comportamiento esperado:

Si la autenticación es exitosa y el usuario está activo, responde con:

Plain Text

```
{  
  "success": true,  "message":  
    "Login exitoso",  
  "data": {}  
}
```

Errores comunes:

- **400** si faltan los campos `email` o `contrasena`.
- **401** si las credenciales son inválidas (email o contraseña incorrectos).
- **403** si el usuario se encuentra en estado distinto a `ACTIVO`

Body raw (json)

json

```
{  
  "email": "admin@opticom.com",  
  "contrasena": "Admin123!"  
}
```

CONTACTO

Esta carpeta agrupa los endpoints relacionados con el módulo de contacto, permitiendo gestionar las solicitudes o mensajes enviados desde esta sección del sistema.

POST Enviar mensaje

`:/api/contacto`

Permite enviar un mensaie de contacto en formato .JSON.

Método y ruta: `POST :/api/contacto`

StartFragment

Body requerido:

Plain Text

```
{
  "nombre": "Juan Pérez",
  "email": "juan.perez@email.com",
  "telefono": "3001234567",
  "mensaje": "Hola, necesito información sobre el servicio."
}
```

Comportamiento esperado:

Si la validación es correcta, responde con:

Plain Text

```
{
  "success": true,
  "message": "Mensaje enviado! Te contactaremos pronto."
}
```

Errores comunes:

- **400** si faltan `nombre`, `email` o `mensaje`.
- **400** si `email` es inválido.

Body raw (json)

json

```
{
  "nombre": "Saida Roza",
  "email": "saidarozo@gmail.com",
  "teléfono": "3228951978",
  "mensaje": "Deseo consultar el costo de lentes según formula"
}
```

GET `raiz`

`:/api/health`

